

FOODLINK SMART FOOD DONATION AND DISTRIBUTION PLATFORM

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr Juttuka Lovababu, APSSDC

Submitted by

ANNAMNEEDI SATYANARAYANA, 23221A0509, BVC ENGINEERING COLLEGE, (Batch -1)

DURGA SURYA TEJA, 23221A0535, BVC ENGINEERING COLLEGE, (Batch -1)

CHITTINEEDI SRIRAM, 23221A0529, BVC ENGINEERING COLLEGE, (Batch -1)

KOMMURI VENKATESHWARA RAO, 23221A0562, BVC ENGINEERING COLLEGE, (Batch -1)

ADABALA ADITHYA SRIMANIHAS, 23221A0502, BVC ENGINEERING COLLEGE, (Batch -1)

KATTOJU S N D M SANTHOSH, 23221A0559, BVC ENGINEERING COLLEGE, (Batch -1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala, my supervisor and mentor, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Annamneedi Satyanarayana, 23221A0509, annamneedisatyanarayana1114@gmail.com

Durga Surya Teja, 23221A0535, suryatejadurga2@gmail.com

Chittineedi Sriram, 23221A0529, sriramchittineedi143@gmail.com

Kommuri Venkateshwara Rao, 23221A0562, kommurivenkatesh1891@gmail.com

Adabala Aditya Sri Manihas, 23221A0502, manihas96@gmail.com

Kattoju S N D M Santosh, 23221A0559, kattojusanthosh05@gmail.com

ABSTRACT

Food wastage and food insecurity continue to be major challenges in many communities. While restaurants, households, and businesses often have surplus edible food, many nongovernmental organizations (NGOs) struggle to obtain food for timely distribution. **FoodLink** is a cloud-based Smart Food Donation and Distribution Platform developed to bridge this gap by connecting food donors with NGOs through a secure and user-friendly web application. The platform provides separate role-based workflows for donors and NGOs. Donors can register, log in, create food donation listings, upload food images, provide pickup details, and track the status of their donations. NGOs can browse available donations, view detailed information, and accept suitable donations. Once a donation is accepted, the application updates the donation status, ensuring transparency and real-time synchronization between both users.

FoodLink is deployed on **Amazon Web Services (AWS)** using **Amazon EC2** as the hosting environment. **Amazon DynamoDB** stores user and donation records, while **Amazon S3** manages uploaded food images. **Amazon SNS** supports notification services, and **Amazon CloudWatch** monitors system performance. The application is developed using **Node.js**, with **Nginx** acting as a reverse proxy and **PM2** managing application processes to ensure reliable operation.

Functional testing verified the successful implementation of donor registration, NGO registration, authentication, donation creation, image upload, donation browsing, and donation acceptance. The project demonstrates the practical application of AWS Cloud Computing and DevOps technologies in building a scalable and reliable food donation platform. FoodLink provides a strong foundation for future enhancements such as location-based donation matching, mobile applications, analytics, and automated pickup management.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION.....	5
2.	METHODOLOGY.....	6
3.	TECHNOLOGY STACK.....	8
4.	SYSTEM DESIGN / ARCHITECTURE	10
5.	IMPLEMENTATION.....	18
6.	RESULTS.....	24
7.	CONCLUSION	30
8.	REFERENCES.....	31

1. INTRODUCTION

In the digital era, cloud computing has become a powerful foundation for building applications that require reliable hosting, scalable data storage, centralized management, and continuous availability. At the same time, food wastage remains a significant social and operational challenge. Large quantities of edible surplus food from households, restaurants, events, and businesses may go unused because donors do not have a direct and organized mechanism to connect with nearby NGOs. FoodLink: Smart Food Donation and Distribution Platform addresses this gap by providing a web-based system that connects food donors with NGOs. Instead of depending on phone calls, informal messaging groups, or manual coordination, donors can create structured donation listings containing food name, category, vegetarian or non vegetarian type, quantity, city, pickup time, address, contact details, and an optional image. NGOs can browse available donations and accept suitable listings. The application uses a rolebased workflow. Donors and NGOs register separately and receive dashboards relevant to their activities. Donors can create and monitor donations, while NGOs can discover available food and accept a donation. The system updates the donation status from Pending to Accepted, allowing the donor to see the latest state through the dashboard. The project adopts a cloudhosted architecture using Amazon Web Services (AWS). Amazon EC2 hosts the Node.js application. Amazon DynamoDB provides NoSQL storage for users and donations. Amazon S3 stores uploaded donation images. Amazon SNS is integrated for notification support, while Amazon CloudWatch monitors EC2 metrics. An Elastic IP provides a stable public address. Nginx routes web traffic to the application, and PM2 keeps the Node.js process running. This report presents the methodology, technology stack, system architecture, AWS service integration, implementation process, testing results, and future scope of FoodLink. The project

demonstrates how cloud and DevOps technologies can be combined to solve a practical social problem through a reliable and maintainable web platform.

2. METHODOLOGY

The development of **FoodLink** followed a structured approach to ensure the successful completion of the project. The work was carried out in different phases, including requirements gathering, system design, application development, database design, AWS service integration, testing, deployment, and final improvements. Each phase helped build a reliable, secure, and efficient cloud-based food donation platform.

2.1. Requirements Gathering

The project began by identifying the core problem: surplus food donors often lack a structured channel to reach NGOs, and NGOs may not have real-time visibility into available donations. The functional requirements were defined around two major user roles — Donor and NGO..

- Donor registration and login.
- NGO registration and login.
- Role-based dashboards.
- Creation of food donation listings.
- Optional donation image upload.
- Browsing and filtering of available donations.
- Viewing detailed donation information.
- NGO acceptance of donations.
- Donation status tracking for donors.

2.2. Design

A web-based, multi-component cloud architecture was designed. User requests enter through the public web endpoint and are handled by Nginx on an EC2 instance. Nginx forwards application traffic to the Node.js server managed by PM2. The application communicates with DynamoDB for structured data, S3 for image objects, and SNS for notifications..

2.3. Implementation

The application was implemented using Node.js and JavaScript. Separate repository modules were used for user and donation data operations. AWS database adapter logic handles DynamoDB interactions. User email addresses are normalized before storage and lookup to improve consistency.

2.4. Database Management

Amazon DynamoDB was selected as the primary database. Two tables were used: foodlink_users and foodlink_donations. Both use an id string attribute as the partition key. The users table also contains the EmailIndex Global Secondary Index, which allows efficient queries using the normalized email attribute.

2.5. Testing

End-to-end functional testing was performed using a complete donor-to-NGO workflow. A donor account and NGO account were registered. The donor logged in and created a donation. The donation was verified in DynamoDB and the uploaded image was verified in S3. The NGO logged in, browsed available donations, opened the donation details, and accepted the donation. The updated Accepted status was then verified on the donor side.

2.6. Deployment

The application was deployed to an Ubuntu-based Amazon EC2 instance. Nginx was configured as a reverse proxy, and PM2 was used to manage the Node.js process. An Elastic IP was associated with the instance to maintain a stable public endpoint. CloudWatch metrics were used to observe the EC2 instance. During testing, configuration and startup issues were identified and corrected iteratively until the full workflow operated successfully.

2.7. Iterative Refinement

Throughout the development of **FoodLink**, the application was continuously tested and improved based on the results obtained during each phase. Issues related to user interface, database operations, AWS service integration, and application deployment were identified and resolved to enhance the overall performance and reliability of the system. Necessary modifications were made to improve usability, optimize application workflows, and ensure smooth communication between donors and NGOs. This iterative process helped develop a stable, secure, and user-friendly food donation platform.

The methodology adopted for the development of this serverless inventory management system ensured a cloud-native, scalable, and efficient solution. By following an event-driven and modular approach, the project successfully demonstrates the practical application of serverless technologies in solving real-world inventory challenges.

3. TECHNOLOGY STACK

FoodLink was developed using a combination of web technologies, AWS cloud services, and DevOps deployment tools.

3.1. Cloud Platform

- **Amazon Web Services (AWS):** Cloud platform used for compute, database, object storage, notifications, networking, and monitoring

3.2. AWS Services

- **Amazon EC2:** Hosts the FoodLink Node.js web application on an Ubuntu server.
- **Elastic IP:** Provides a stable public IPv4 address for the deployed application.
- **Amazon DynamoDB:** Stores user and donation records in NoSQL tables.
- **DynamoDB Global Secondary Index:** EmailIndex supports efficient email-based user queries.
- **Amazon S3:** Stores uploaded donation images as cloud objects.
- **Amazon SNS:** Provides a publish/subscribe notification mechanism for application events.
- **Amazon CloudWatch:** Monitors EC2 performance and resource metrics.
- **EC2 Security Groups:** Control inbound traffic for SSH, HTTP, HTTPS, and application access.

3.3. Programming & Web Technologies

- **Node.js:** JavaScript runtime used for the server-side application.
- **JavaScript:** Implements application logic and AWS integration.
- **HTML and CSS:** Used for the web interface and responsive pages.

- **AWS SDK / AWS CLI:** Used to interact with AWS services and validate cloud resources.

3.4. Data Storage

- **Amazon DynamoDB:** Used as the primary NoSQL database to store donor, NGO, and food donation records.
- **Amazon S3:** Used to securely store food donation images uploaded by donors.

3.5. Security & Access

- **EC2 Security Groups:** Configured to control inbound and outbound traffic, allowing secure access to the application.
- **AWS Credentials:** Configured on the EC2 instance to enable secure communication with AWS services such as DynamoDB, S3, SNS, and CloudWatch.

3.6. Monitoring & Logging

- **Amazon CloudWatch Metrics:** Used to monitor the performance and health of the Amazon EC2 instance, including CPU utilization, network activity, and system statusCloudWatch
- **PM2 Logs:** Used to monitor the Node.js application, capture runtime logs, and troubleshoot application errors.

4. SYSTEM DESIGN / ARCHITECTURE

FoodLink is designed as a cloud-hosted web application that integrates compute, database, object storage, notification, and monitoring services. The architecture is centered on an Amazon

EC2 instance running the application stack. Donors and NGOs access the system using a web browser through the application's Elastic IP

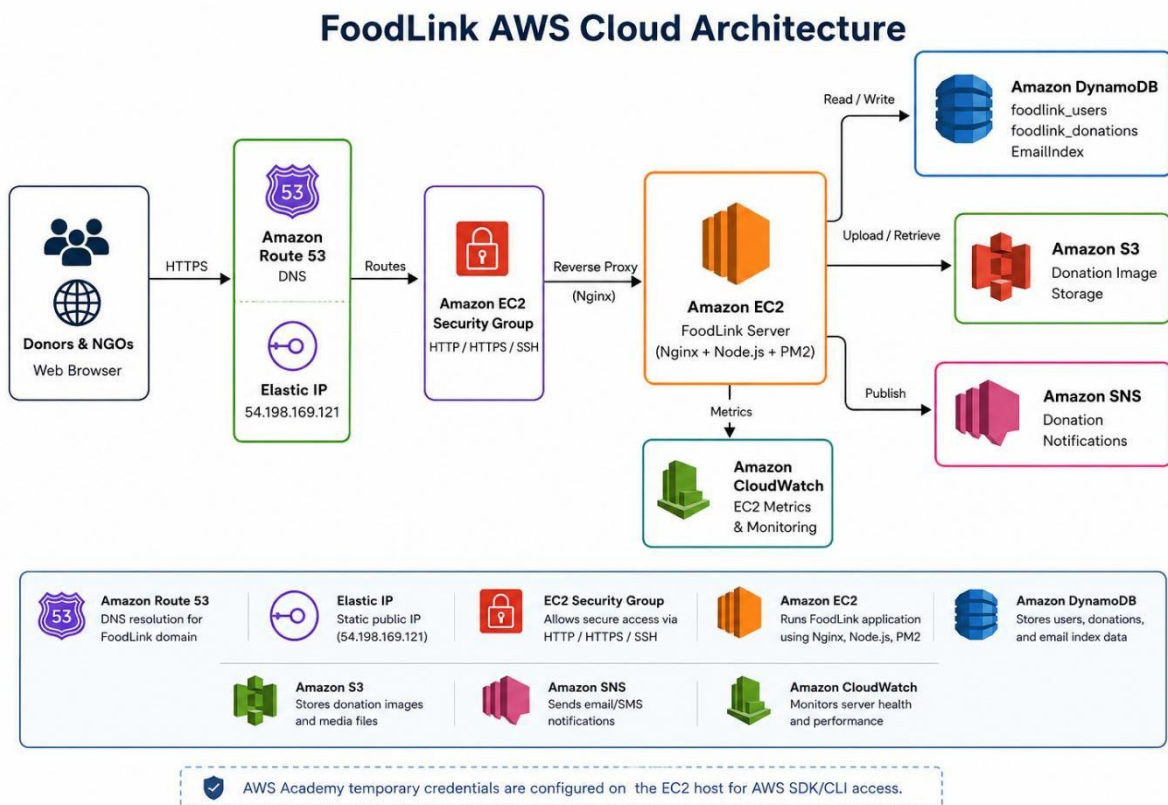


Figure 4.1: FoodLink AWS Cloud Architecture

4.1. User Access and Public Endpoints

Donors and NGOs access FoodLink through a browser. The Elastic IP associated with the EC2 instance provides a stable public IPv4 address. This avoids depending on a changing autoassigned public IP during the deployment lifecycle.

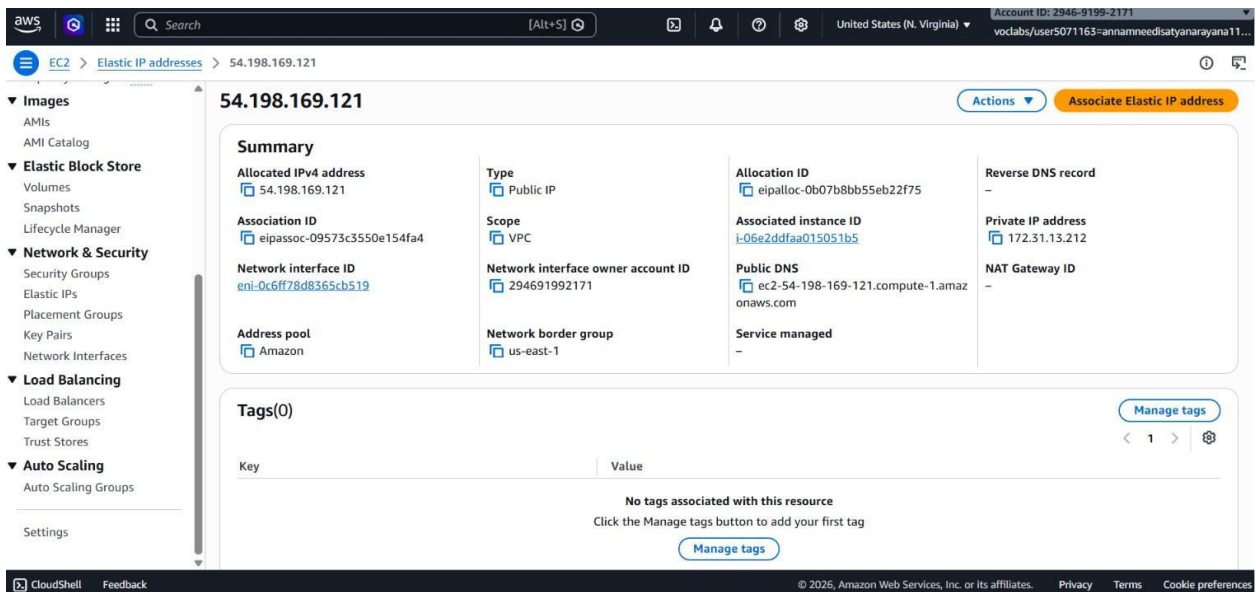


Figure 4.2: Elastic IP Associated with the FoodLink Deployment

4.2. Compute Layer – Amazon EC2

- Amazon EC2 provides the virtual server used to host FoodLink. The Ubuntu instance runs the Node.js application, PM2 process manager, and Nginx web server. The EC2 instance processes user registration, login, donation creation, donation browsing, acceptance, and dashboard requests

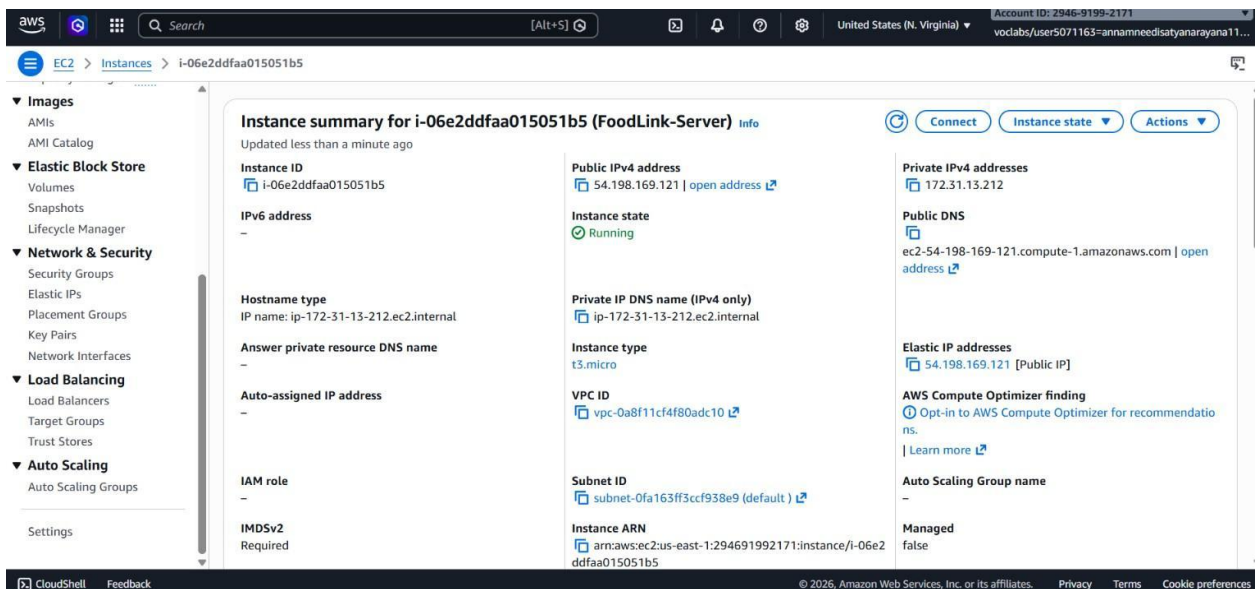


Figure 4.3: Amazon EC2 Instance Used for FoodLink

4.3. Security Layer – EC2 Security Group

The EC2 Security Group acts as a virtual firewall for the server. Inbound rules were configured to support SSH administration and web access. During deployment, application-related ports were configured as required for testing and Nginx-based public access.

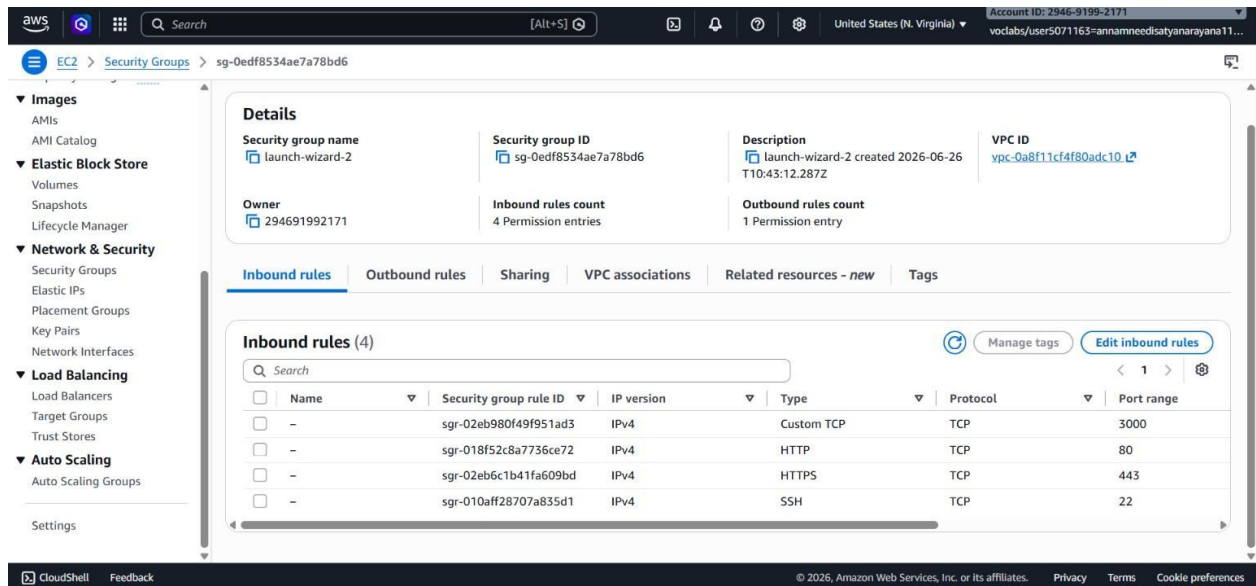


Figure 4.4: EC2 Security Group Inbound Rules

4.4. Database Layer – Amazon DynamoDB

Amazon DynamoDB is used as the primary NoSQL database for the FoodLink application. It stores all user accounts and food donation records in a secure and scalable manner. The **foodlink_users** table maintains donor and NGO information, while the **foodlink_donations** table stores details such as food name, category, quantity, pickup location, donation status, and donor information. Both tables use the **id** attribute as the partition key. To improve login performance, the **foodlink_users** table includes a **Global Secondary Index (EmailIndex)** that enables efficient email-based user lookup during authentication.

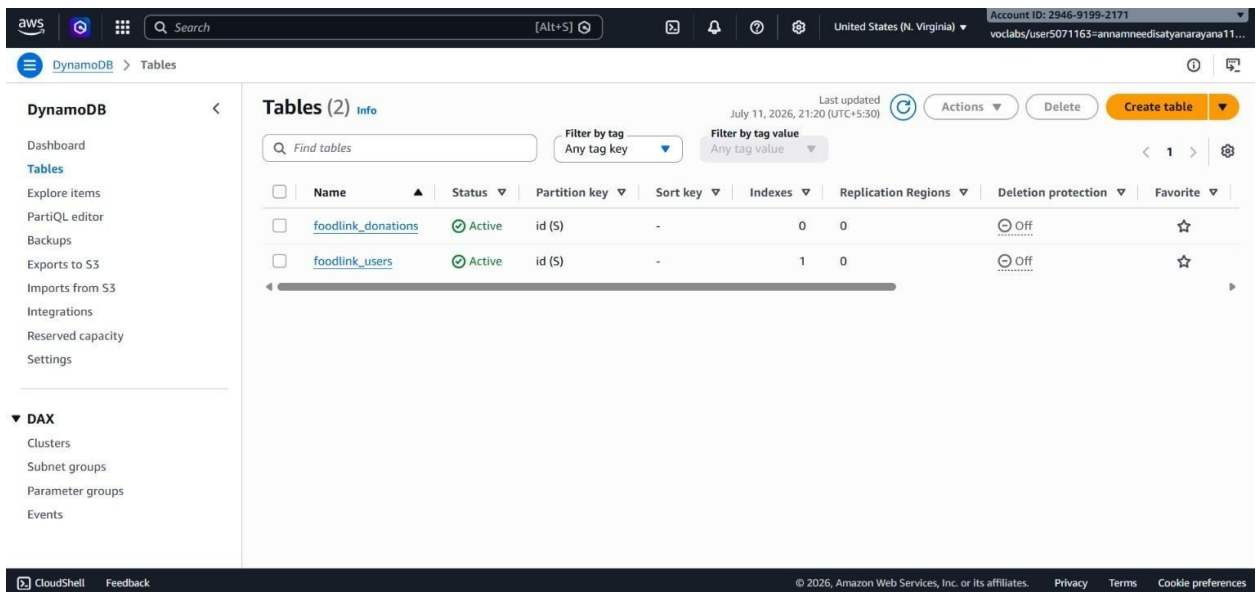


Figure 4.5: FoodLink DynamoDB Tables

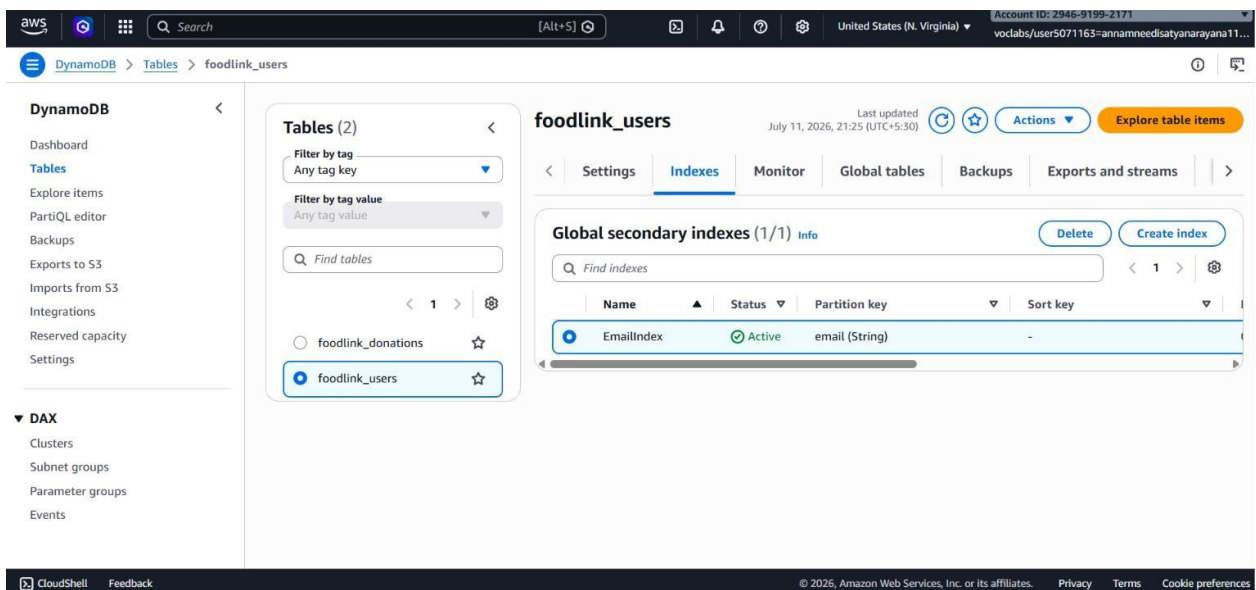


Figure 4.6: EmailIndex Global Secondary Index

4.5. Object Storage Layer – Amazon S3

- Amazon S3 is used to securely store food donation images uploaded by donors. When a donor creates a donation with an image, the application uploads the image to an S3 bucket and stores the image reference along with the donation record in DynamoDB. Using Amazon S3 provides reliable, scalable, and highly available storage while reducing the load on the application server.

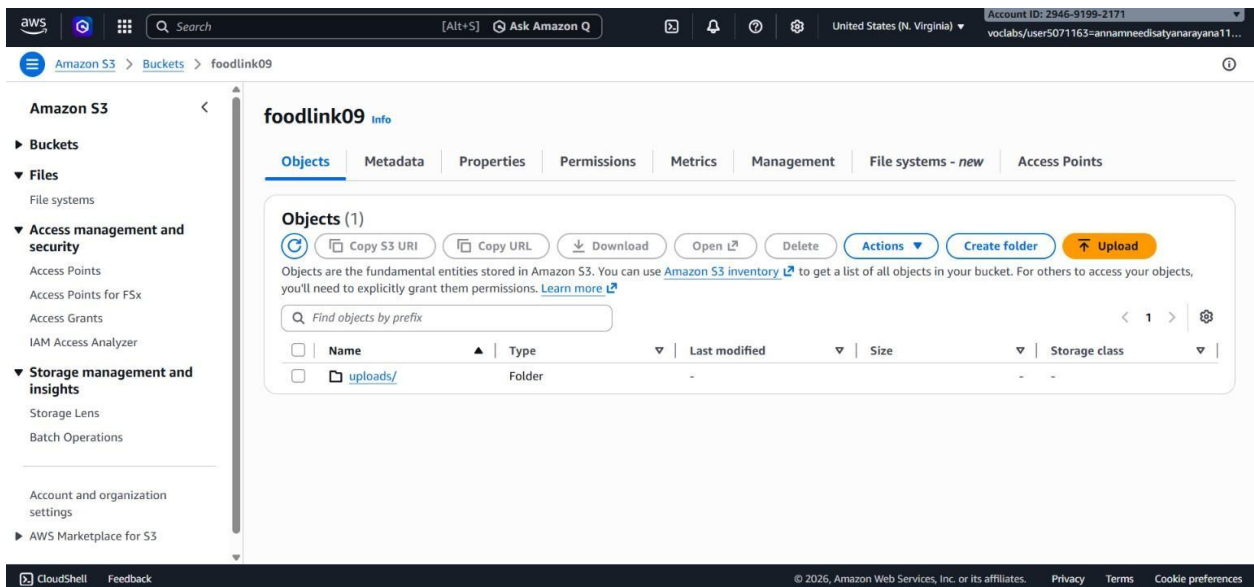


Figure 4.7: Amazon S3 Bucket Used for Donation Images

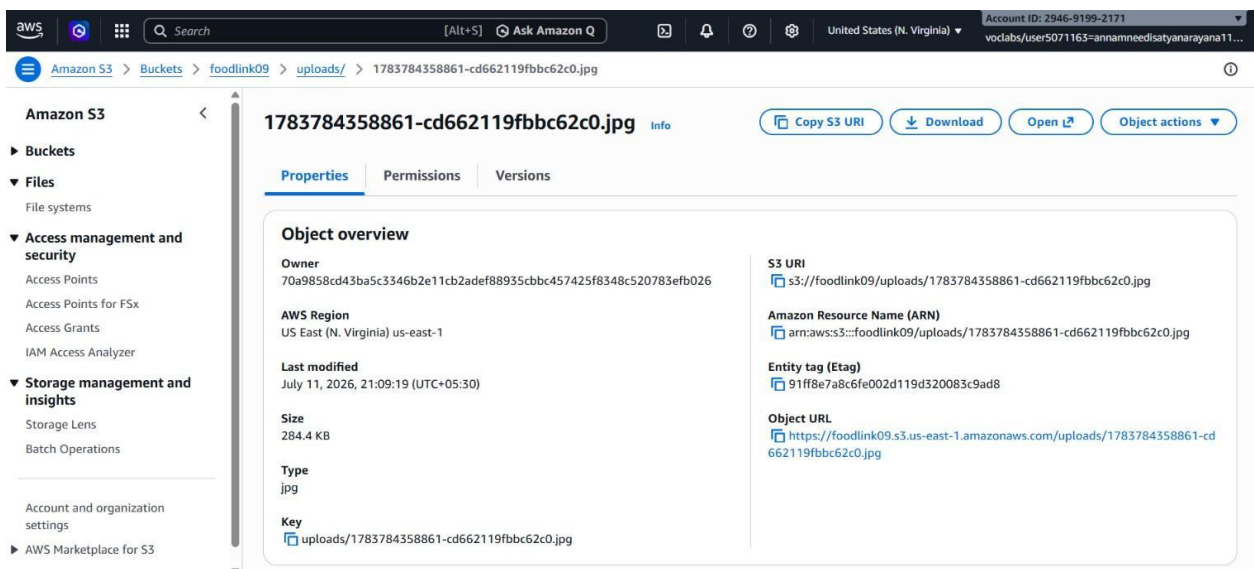


Figure 4.8: Uploaded Donation Image Object Details in S3

4.6. Notification Layer – Amazon SNS

- Amazon Simple Notification Service (SNS) is integrated into FoodLink to support notification services. An SNS topic and subscription were configured to enable the delivery of application-related notifications. This service can be used to notify users about important donation events and provides a scalable publish/subscribe communication mechanism.

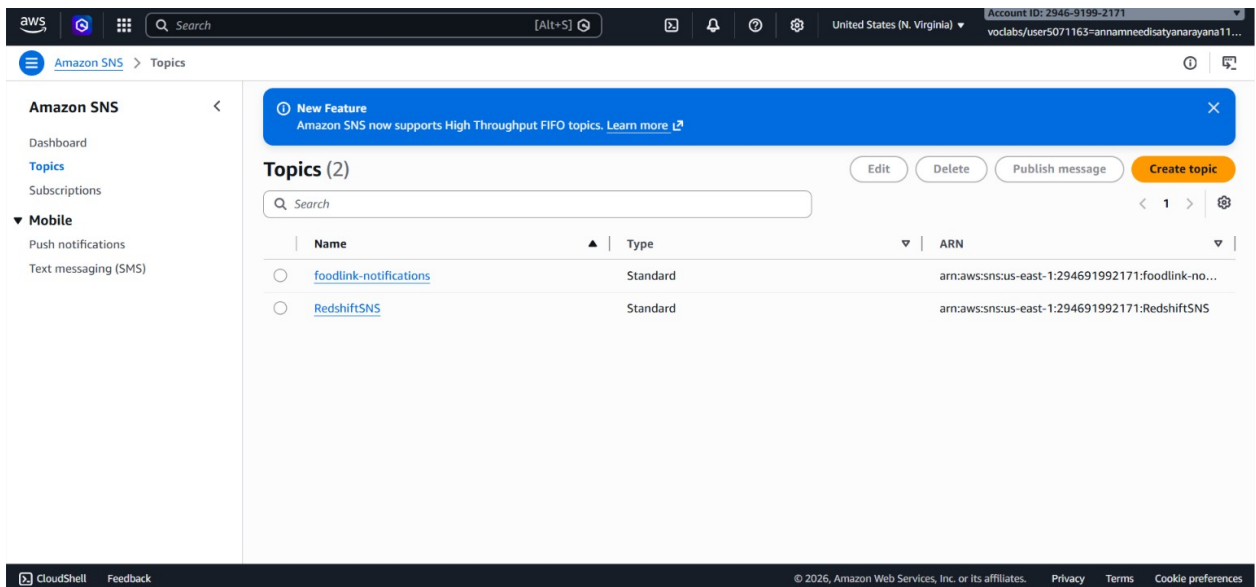


Figure 4.9: Amazon SNS Topic for FoodLink Notifications

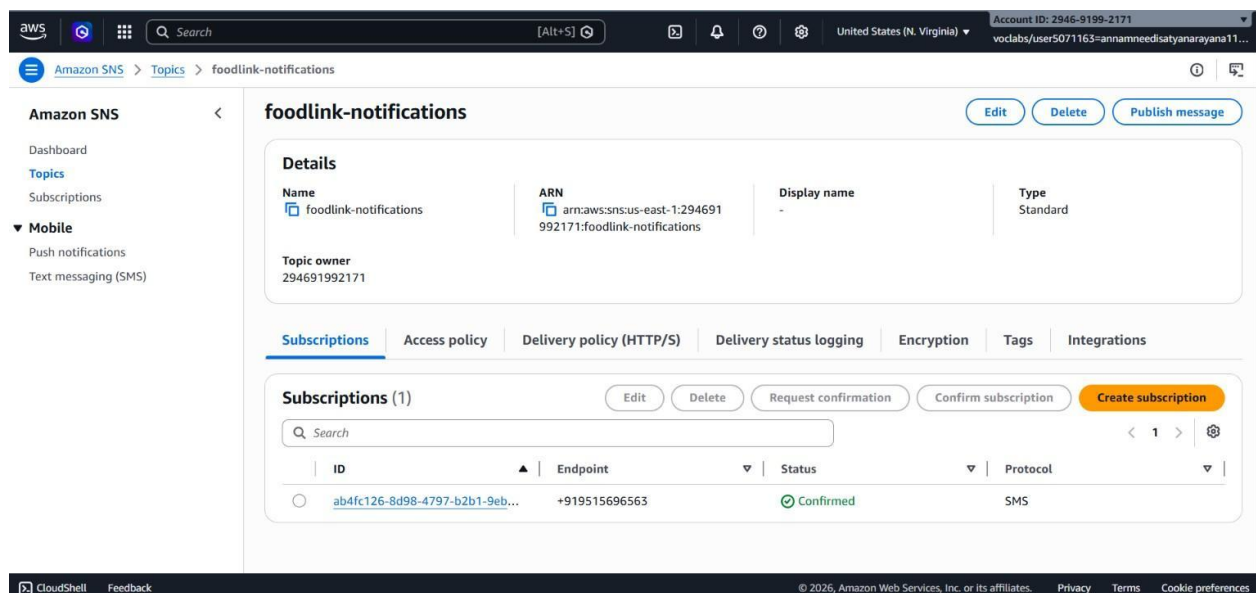


Figure 4.10: SNS Subscription Configuration

4.7. Monitoring Layer – Amazon CloudWatch

- Amazon CloudWatch is used to monitor the performance and health of the FoodLink application hosted on Amazon EC2. It provides real-time metrics such as CPU utilization, network traffic, and instance status, helping administrators monitor system performance and identify potential issues during application execution.

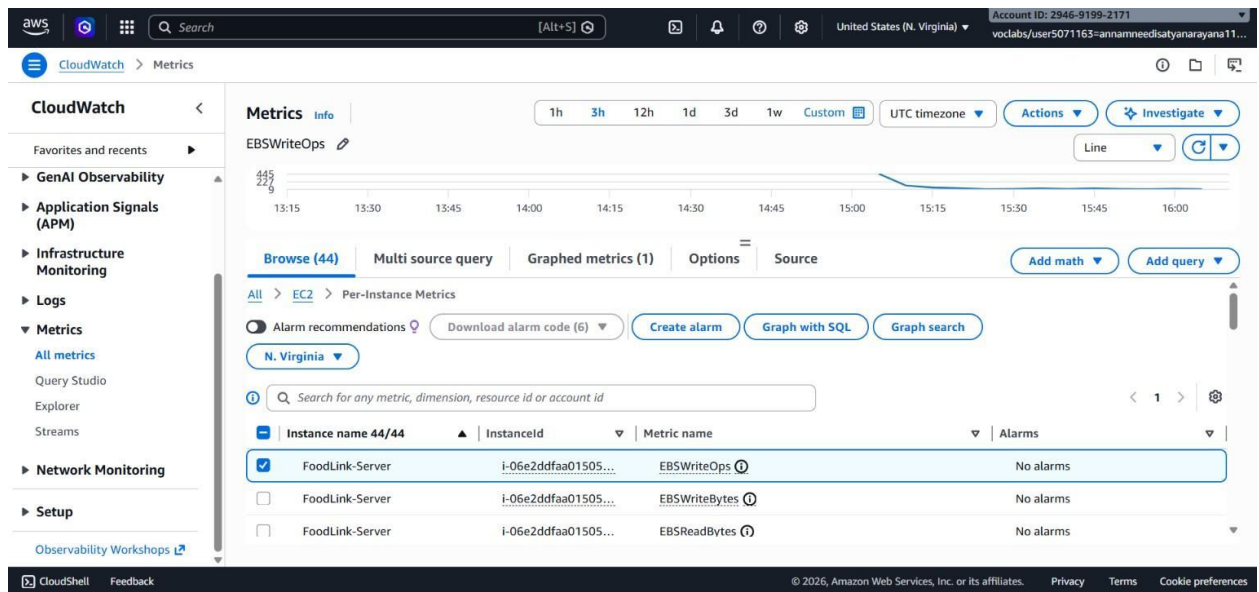


Figure 4.11: Amazon CloudWatch EC2 Metrics

4.8. Application Request Flow

The workflow of the FoodLink application is as follows:

1. A donor or NGO accesses the FoodLink application using the public Elastic IP address.
2. The EC2 Security Group allows incoming HTTP requests based on the configured security rules.
3. Nginx receives the client request and forwards it to the Node.js application.
4. PM2 manages the Node.js application and ensures continuous service availability.
5. The application processes user requests and stores or retrieves data from Amazon DynamoDB.
6. Food donation images are uploaded to and retrieved from Amazon S3 whenever required.
7. Notification events are published through Amazon SNS for application communication.
8. Amazon CloudWatch continuously monitors the EC2 instance and application performance.

5. IMPLEMENTATION

The FoodLink implementation translates the architecture into a working cloud-based food donation workflow. The application was configured and deployed on EC2, integrated with DynamoDB, S3, and SNS, and made publicly accessible using Nginx and an Elastic IP.

5.1. User Repository and DynamoDB Integration

The FoodLink application uses Amazon DynamoDB to manage user information and authentication. During user registration, each donor and NGO is assigned a unique ID, and email addresses are stored in a standardized format to ensure consistent user lookup. The **EmailIndex** Global Secondary Index is used to retrieve user records efficiently during the login process, improving authentication performance without scanning the entire database.

5.2. DynamoDB Table Configuration

Two DynamoDB tables were created to store application data. The `foodlink_users` table stores donor and NGO account information, while the `foodlink_donations` table stores food donation details, pickup information, and donation status. Both tables use On-Demand (PAY_PER_REQUEST) billing mode, which automatically scales according to application usage without requiring manual capacity management.

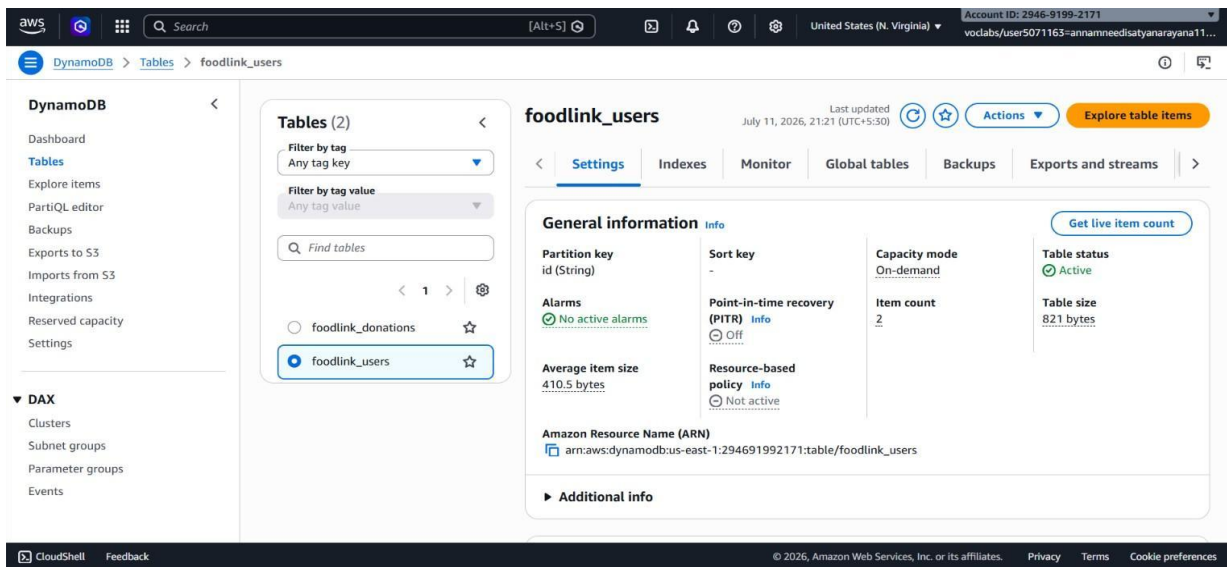


Figure 5.1: foodlink_users Table Configuration

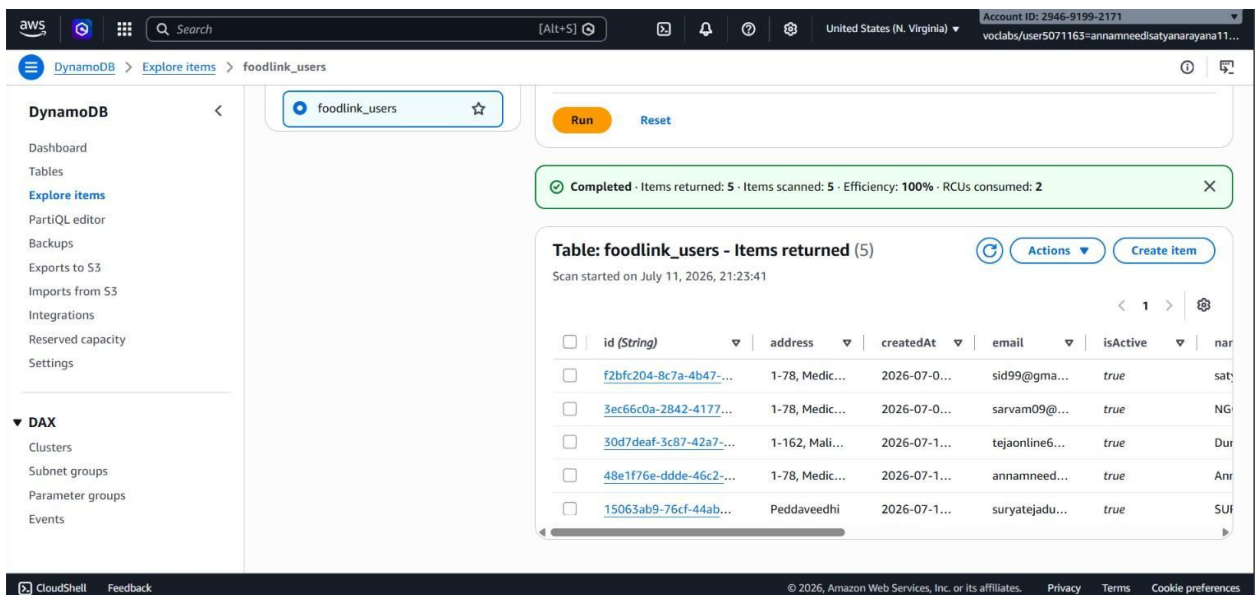


Figure 5.2: Donor and NGO Records Stored in foodlink_users

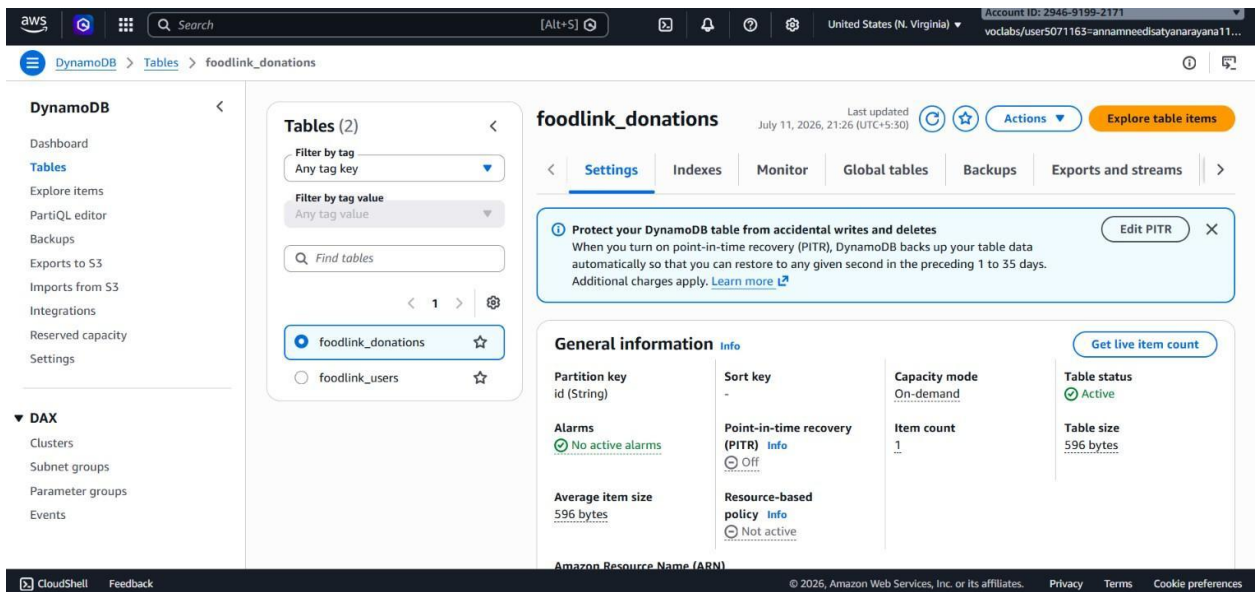


Figure 5.3: foodlink_donations Table Configuration

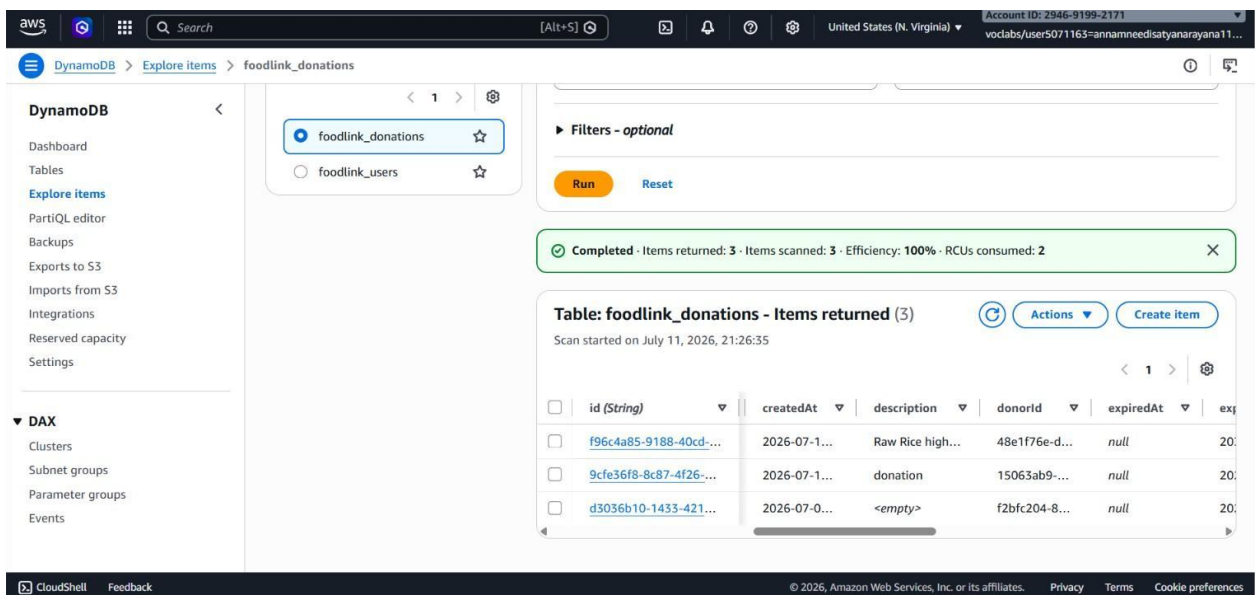


Figure 5.4: Donation Records Stored in DynamoDB

5.3. Amazon S3 Image Upload Integration

FoodLink allows donors to upload images while creating food donation listings. The uploaded images are stored securely in an Amazon S3 bucket, while the corresponding image reference is

saved in the DynamoDB donation record. This approach provides scalable storage, improves application performance, and ensures reliable access to uploaded images.

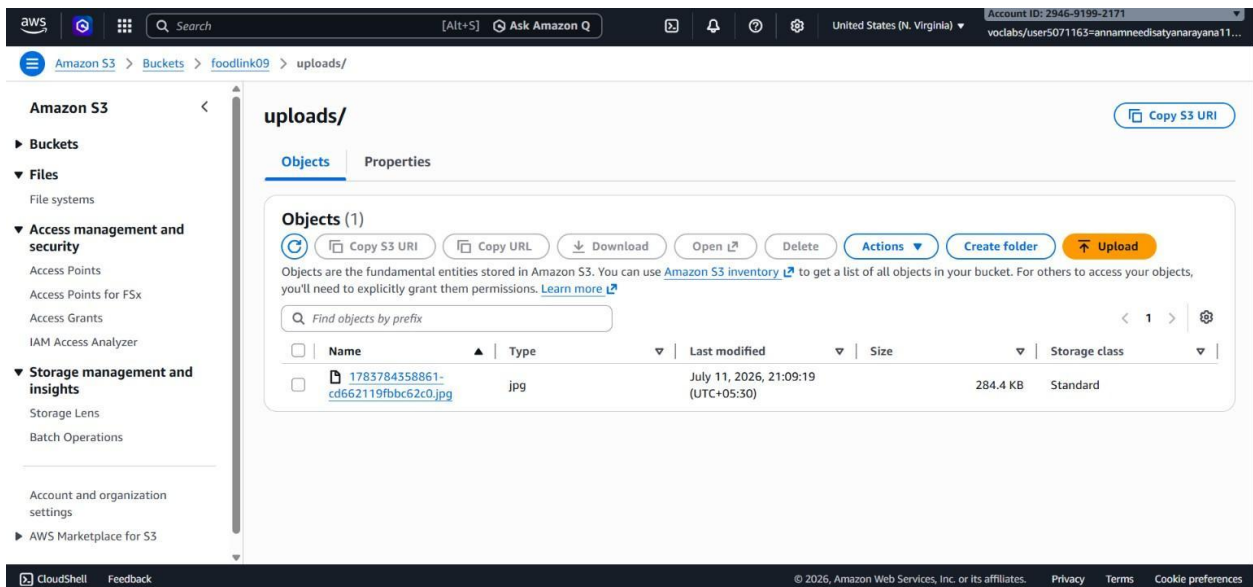


Figure 5.5: Donation Image Object Stored in Amazon S3

5.4. Amazon SNS Notification Configuration

Amazon SNS was configured to support the notification functionality within the application. An SNS topic and subscription were created through the AWS Console, allowing the application to publish notification events whenever required. This configuration provides a scalable communication mechanism for future notification enhancements.

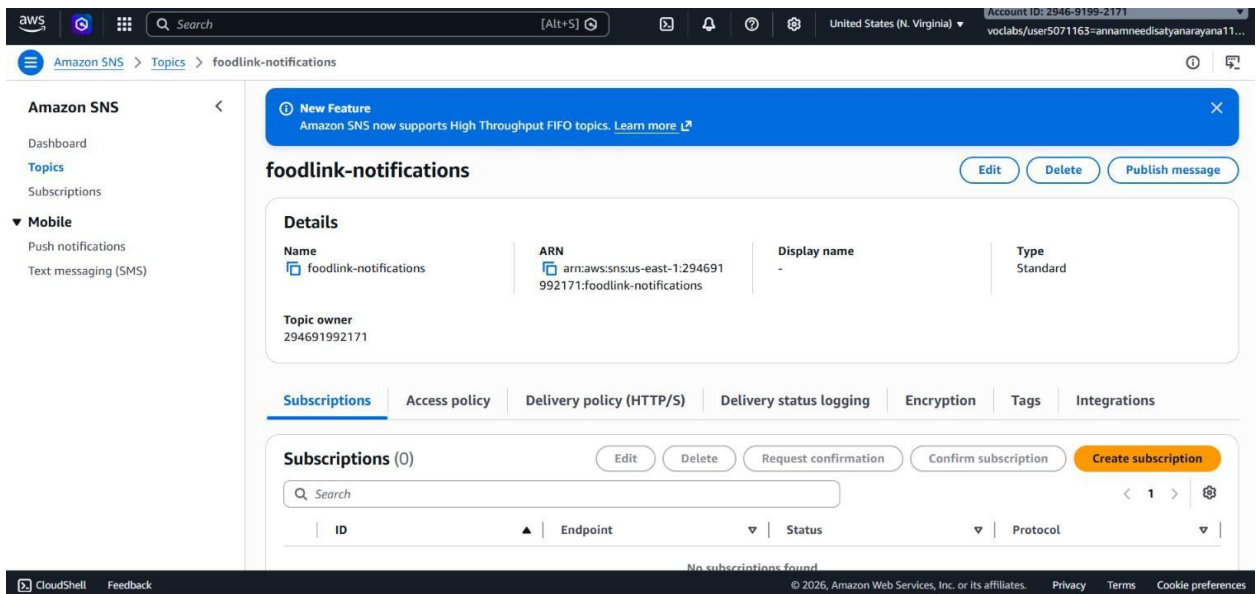


Figure 5.6: FoodLink SNS Topic Details

5.5. Application Process Management with PM2

- The FoodLink Node.js server is managed using PM2. The process was started with the name foodlink and verified as online. The PM2 process list was saved so the process definition could be restored after server or lab restarts.

```

ubuntu@ip-172-31-13-212: ~$ ssh -i "FoodLink-Key.pem" ubuntu@54.198.169.121
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 11 16:11:27 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   43.9% of 6.61GB Processes:      124
Memory usage: 36%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.13.212

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat Jul 11 15:13:13 2026 from 103.99.8.169
ubuntu@ip-172-31-13-212:~$ cd ~/FoodLink
pm2 status

```

id	name	namespace	version	mode	pid	uptime	o	status	cpu	mem	user	watching
0	foodlink	default	1.0.0	fork	1027	66m	0	online	0%	185.2mb	ubuntu	disabled

```

ubuntu@ip-172-31-13-212:~/FoodLink$

```

Figure 5.7: FoodLink Application Online in PM2

5.6. Nginx Reverse Proxy Configuration

- Nginx receives web requests on the public endpoint and forwards them to the Node.js application. The Nginx service status and reverse proxy configuration were verified on the EC2 instance.

```
ubuntu@ip-172-31-13-212: ~$ sudo apt update
Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat Jul 11 15:13:13 2026 from 103.99.8.169
ubuntu@ip-172-31-13-212:~$ cd ~/FoodLink
pm2 status
```

id	name	namespace	version	mode	pid	uptime	o	status	cpu	mem	user	watching
0	foodlink	default	1.0.0	fork	1027	66m	0	online	0%	105.2mb	ubuntu	disabled

```
ubuntu@ip-172-31-13-212:~/FoodLink$ sudo systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-11 15:05:14 UTC; 1h 7min ago
     Invocation: 193686f5e9364af5bd17a78ccf4607d4
       Docs: man:nginx(8)
    Process: 615 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 659 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 700 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 4.9M (peak: 5.6M)
        CPU: 146ms
    CGroup: /system.slice/nginx.service
            └─700 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─705 "nginx: worker process"
                └─706 "nginx: worker process"

Jul 11 15:05:13 ip-172-31-13-212 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jul 11 15:05:14 ip-172-31-13-212 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-13-212:~/FoodLink$
```

Figure 5.8: Nginx Service Running on EC2

5.7. Deployment Verification

- After associating the Elastic IP, starting the FoodLink process, and confirming Nginx operation, the application was accessed through a browser. The FoodLink home page loaded successfully from the public deployment.

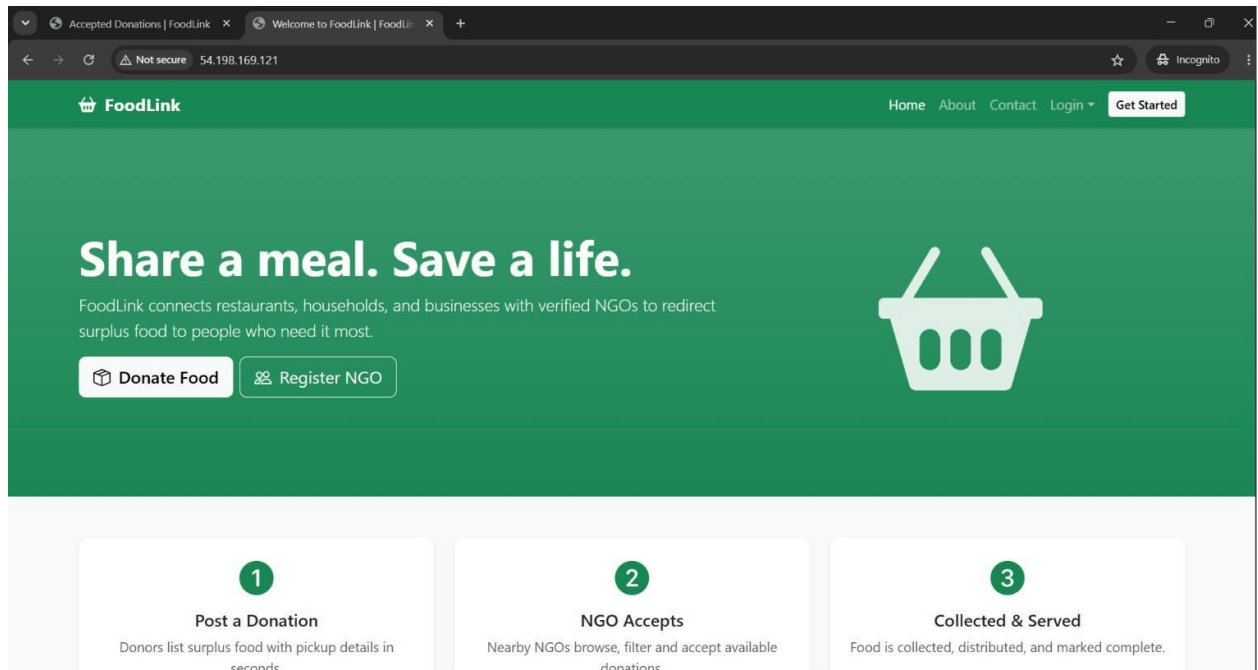


Figure 5.10: FoodLink Successfully Deployed and Publicly Accessible

6. RESULTS

The implementation and deployment of FoodLink produced a fully functional donor-to-NGO food donation workflow. The system was tested through the web interface and AWS console to verify application behavior and cloud data persistence.

6.1. Successful Home Page Deployment

The FoodLink home page was successfully loaded from the EC2-hosted deployment. The interface presents the platform purpose and entry points for donor and NGO workflows.

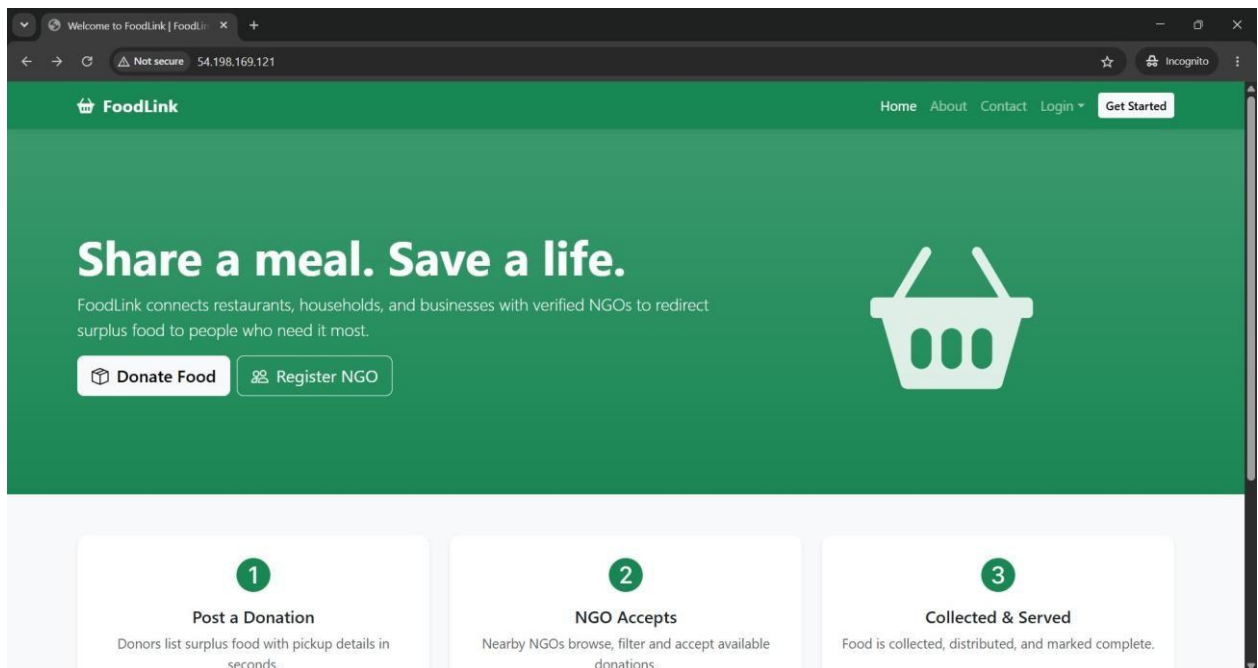


Figure 6.1: FoodLink Home Page

6.2. Donor Registration

- A donor account was successfully registered using the donor registration form. The application accepted user information and stored the user record in DynamoDB

Figure 6.2: Donor Registration Interface

6.3. Donor Dashboard

After authentication, the donor dashboard displayed donation statistics and provided the Create Donation action. The dashboard is designed to reflect total, accepted, and pending donations

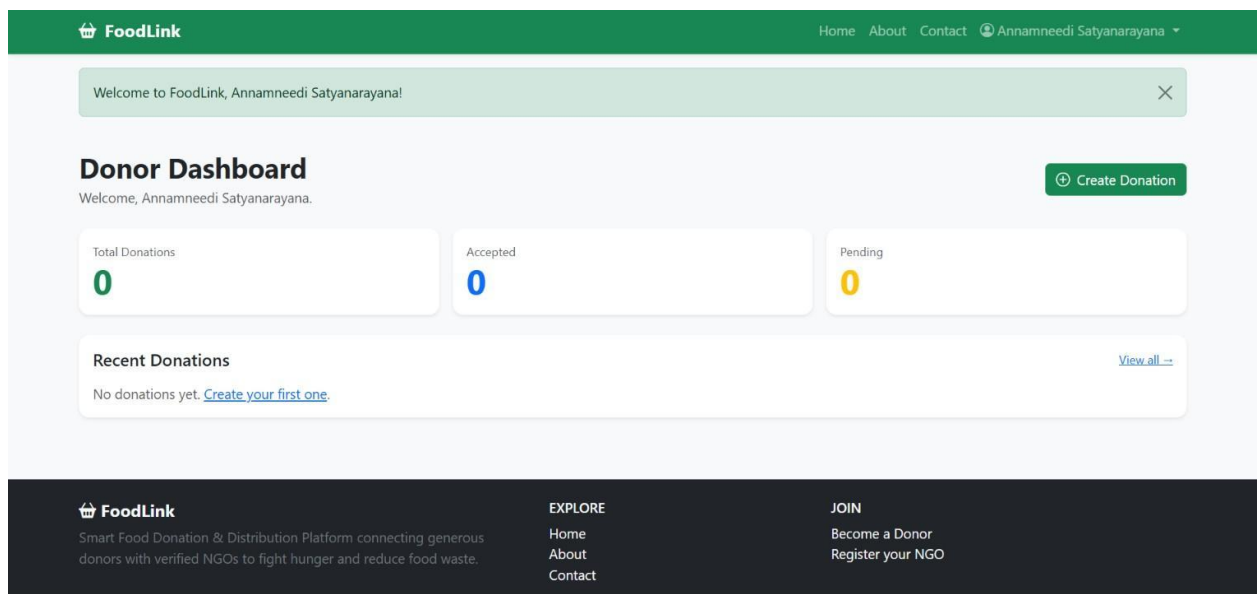


Figure 6.3: Donor Dashboard

6.4. NGO Registration

- An NGO account was registered through a dedicated NGO registration workflow. This establishes a separate role for browsing and accepting donations.

FoodLink Home About Contact Login [Get Started](#)

Register Your NGO

Join FoodLink to receive food donations.

Contact Person Name: Durga Surya Teja

Organization Name: BVCEC

Registration Number: 23221A0535

Email: tejaonline67@gmail.com

Phone: 9666133228

Address: 1-162, Malikipuram, Dr Br.ambedkar konaseema district, And

Password:

Confirm Password:

[Register NGO](#)

[Already registered? Login](#)

Figure 6.4: NGO Registration Interface

6.5. Donation Creation

The donor successfully created a food donation by entering food details, quantity, location, pickup time, contact information, and additional notes. The resulting donation was stored in DynamoDB and became available to the NGO workflow.

FoodLink Home About Contact [Annamneedi Satyanarayana](#)

Create Donation

Food Name *: Rice

Category *: Raw Ingredients

Veg / Non-Veg *: Veg

Quantity *: 15kg

Pickup Time *: 12-07-2026 16:30

Expiry Time *: 31-12-2030 23:59

Pickup Address *: 1-78, Medicharlapalem, Pallapu Veedhi, Malikipuram Mandalam, Dr Br.ambedkar konaseema district, Andhra Pradesh

City: Amalapuram

Contact Number *: 9515696563

Image (optional, max 5 MB): Choose File 106021.jpg

Description: Raw Rice high quality

[Post Donation](#) [Cancel](#)

Figure 6.5: Create Donation Form with Food Details

6.6. Browse Available Donations

The available donations page displayed food listings and provided filters for food name, category, vegetarian/nonvegetarian type, and city. The created donation appeared in the NGO workflow.

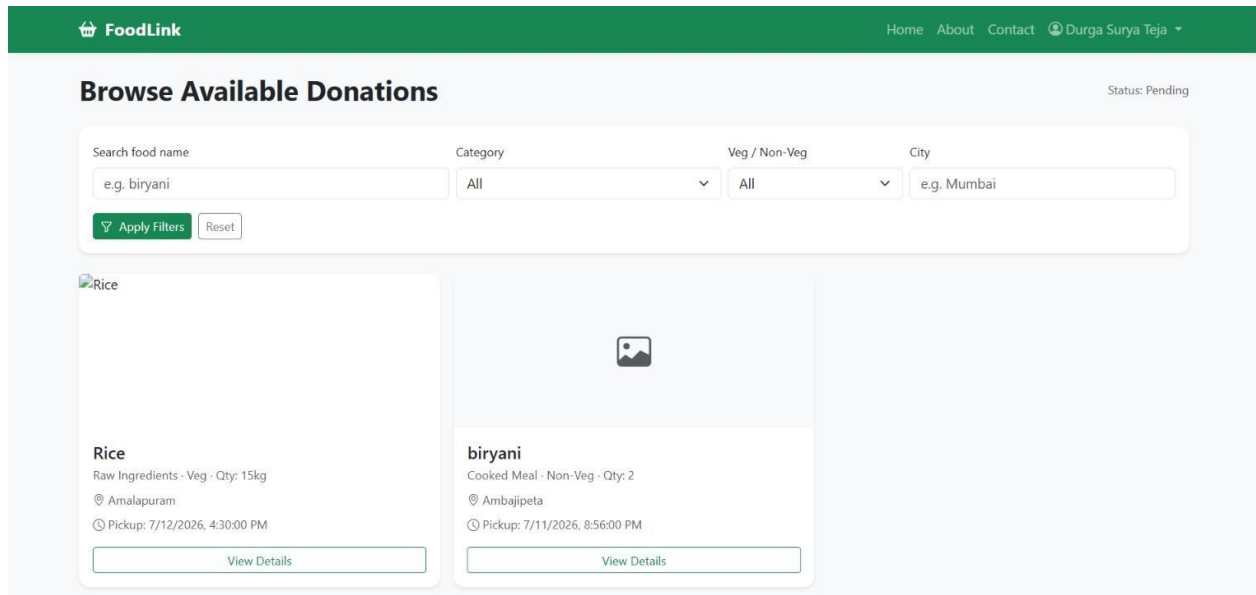


Figure 6.6: Available Donations and Search Filters

6.7. Donation Details

The NGO successfully opened the donation details page. The page displayed food information and pickup-related details required to evaluate the donation.

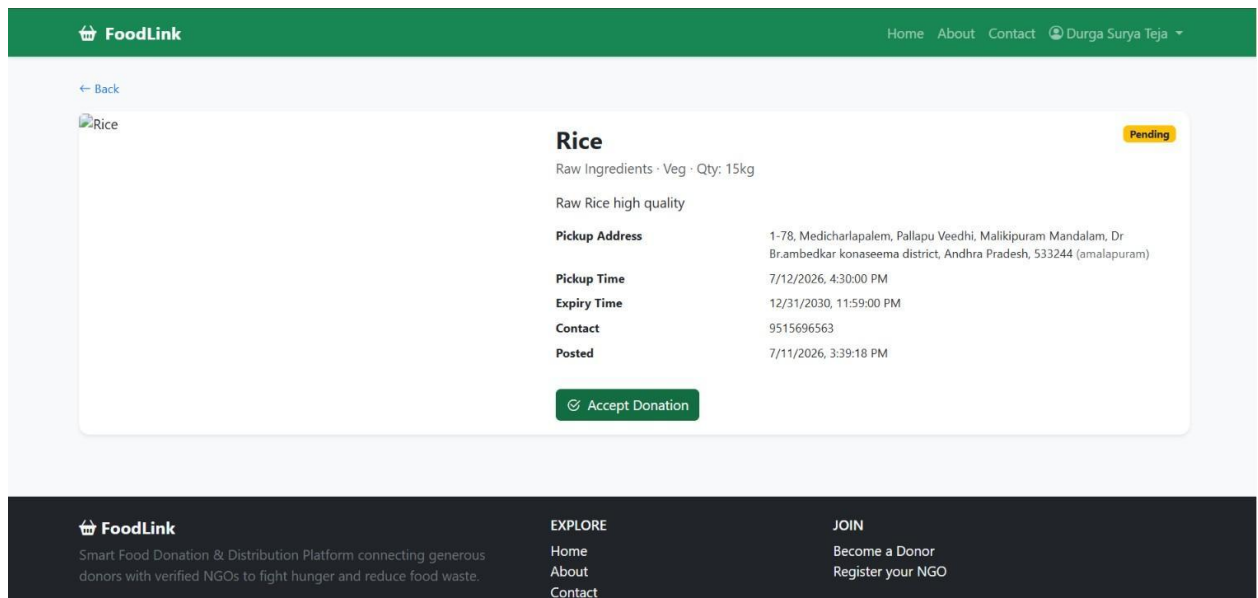


Figure 6.7: Donation Details Before Acceptance

6.8. Donation Acceptance

The NGO accepted the donation successfully. The application updated the donation status and displayed confirmation in the NGO workflow.

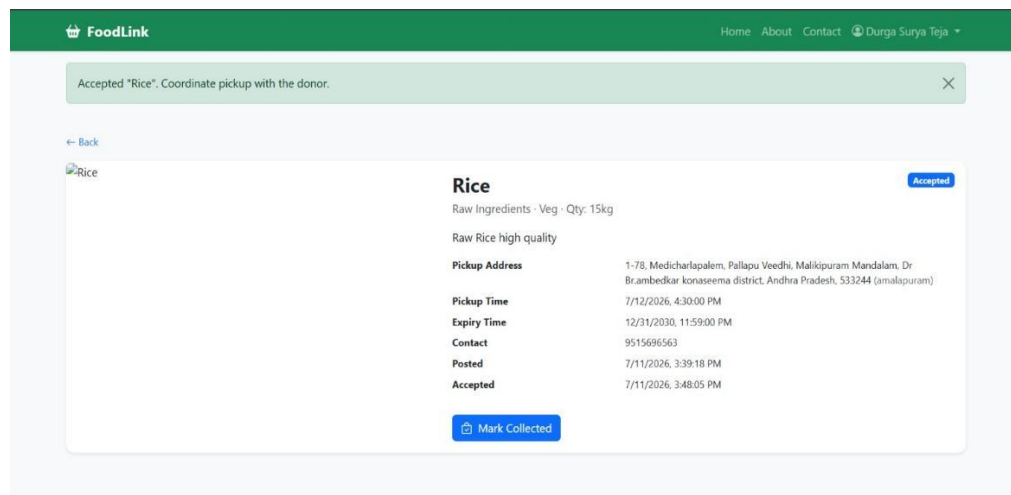


Figure 6.8: Donation Accepted by NGO

7.CONCLUSION

The successful development and deployment of FoodLink demonstrate how AWS cloud services and DevOps tools can be integrated to build a practical food donation and distribution platform. The application connects donors and NGOs through a role-based web workflow and provides a structured process for creating, discovering, and accepting food donations.

Amazon EC2 provides the application hosting environment, while the Elastic IP supplies a stable public endpoint. Amazon DynamoDB stores user and donation data, and the EmailIndex Global Secondary Index supports efficient email-based user lookup. Amazon S3 stores donation images, Amazon SNS provides notification infrastructure, and Amazon CloudWatch supports EC2 monitoring. Nginx and PM2 improve application routing and process reliability on the server.

Testing confirmed that donor registration, NGO registration, login, donation creation, image storage, donation browsing, detail viewing, NGO acceptance, and donor status updates operate successfully. The AWS console evidence also confirms that application data and cloud resources are integrated with the deployed system.

FoodLink provides a strong foundation for future expansion. Possible enhancements include GPS-based nearby donation matching, map integration, verified NGO approval, automated expiry handling, pickup scheduling, OTPbased handover confirmation, push notifications, analytics dashboards, a mobile application, HTTPS with a domain name, CI/CD pipelines, containerized deployment, and fine-grained production IAM roles.

In conclusion, FoodLink successfully applies AWS cloud computing and DevOps concepts to a meaningful realworld problem. The project shows that a cloud-hosted architecture can improve coordination between food donors and NGOs while providing scalable storage, centralized data management, and a maintainable deployment model.

8.REFERENCES

1. Amazon Web Services, "Amazon EC2 Documentation." Available: <https://docs.aws.amazon.com/ec2/>
2. Amazon Web Services, "Amazon DynamoDB Documentation." Available: <https://docs.aws.amazon.com/dynamodb/>
3. Amazon Web Services, "Amazon S3 Documentation." Available: <https://docs.aws.amazon.com/s3/>
4. Amazon Web Services, "Amazon Simple Notification Service (SNS) Documentation." Available: <https://docs.aws.amazon.com/sns/>
5. Amazon Web Services, "Amazon CloudWatch Documentation." Available:
6. <https://docs.aws.amazon.com/cloudwatch/>
7. Amazon Web Services, "Elastic IP Addresses Documentation." Available: <https://docs.aws.amazon.com/elastic-ip/>
8. Amazon Web Services, "AWS CLI Documentation." Available: <https://docs.aws.amazon.com/cli/>
9. Node.js Documentation. Available: <https://nodejs.org/en/docs>
10. PM2 Documentation. Available: <https://pm2.keymetrics.io/docs>
11. Nginx Documentation. Available: <https://nginx.org/en/docs>